



SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)



SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA1402

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. (BL3)

Assessment Tool Weightage: 10%

QUESTIONS:

Total Marks:50

Annexure A

SCENARIO BASED TASK

Code of Conduct:

I AL NAJEEB A (192424325) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

AL NAJEEB A



Annexure A

SCENARIO-BASED TASK

QUESTIONS:

1.Scenario: Parsing Table Conflict in LL(1) While designing an LL(1) parser, a student constructs a parsing table and finds multiple entries in a single cell for a non-terminal and input symbol combination. This leads to confusion during parsing. Analyze how the syntax analyzer uses FIRST and FOLLOW sets in this process, identify the reason for the conflict, and explain how it affects deterministic parsing. Conclude by suggesting what decision should be taken to modify the grammar so that it becomes suitable for LL(1) parsing.

2.Scenario: Incorrect Parse Tree Generation A syntax analyzer generates a parse tree for the expression $id + id - id$, but the evaluation gives incorrect results due to improper grouping of operators. Analyze how the syntax analyzer constructs the parse tree, identify the issue related to associativity, and explain how grammar rules influence the structure of the tree. Finally, discuss what decision should be taken to enforce correct associativity and ensure accurate parsing.

3.Scenario: Invalid Expression Handling A syntax analyzer in a compiler receives the token sequence corresponding to the expression $a + * b$ from the lexical analyzer. Although all tokens are valid individually, the parser fails to generate a parse tree and reports an error. In this situation, analyze how the syntax analyzer identifies the error based on grammar rules, determine the key issue in the token arrangement, and explain how parsing techniques such as predictive or shift-reduce parsing handle such cases. Further, discuss what decision should be taken to improve error detection and recovery while maintaining clarity in parsing.

4. Scenario: Ambiguity in Conditional Statements A syntax analyzer is designed using the grammar $S \rightarrow \text{if } E \text{ then } S \mid \text{if } E \text{ then } S \text{ else } S \mid a$. While constructing the parsing table, conflicts arise, and the parser cannot decide which production to apply for certain inputs. In this scenario, explain how the syntax analyzer encounters ambiguity, identify the key issue known as the dangling else problem, and discuss how it impacts parsing. Suggest an appropriate modification or parsing strategy to resolve the ambiguity and ensure correct association of else clauses.

5.Scenario: Operator Precedence Conflict A parser is designed for arithmetic expressions using the grammar $E \rightarrow E + E \mid E * E \mid id$. While parsing the input $id + id * id$, the syntax analyzer produces an incorrect parse tree due to ambiguity in operator precedence. Examine how the syntax analyzer processes this input, identify the issue caused by the grammar, and explain how precedence and associativity rules should be applied. Based on this, suggest what changes must be made to the grammar or parsing technique to ensure correct evaluation.



RUBRICS FOR CALCULATION:

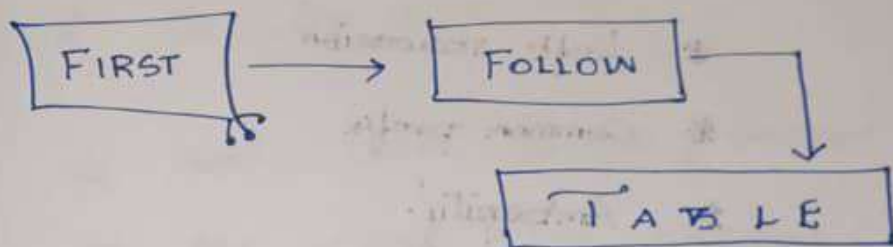
Scenario based assessment

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1. Scenario : Parsing Table Conflict in LL(1)

* Constructing an LL(1) parsing table depends entirely on First and Follow set Computed for the grammar. The first set of a non-terminal lists every terminal that can begin a string derivable

* The parser generation scans each production $A \rightarrow \alpha$ and for every terminal in $\text{First}(\alpha)$ and $\text{follow}(\alpha)$ it can derive empty.



* In the Scenario, the student finds more than one production written into the same cell of the table. This happens when two productions are aligned in a cell.

Discussion :

The grammar must be reworked before it can drive an LL(1) parser. This means removing left recursion by converting recursive production into an equivalent right recursive, left factoring any production that shares a common prefix.

When grammar does not fix how repeated operators of the same precedence nest inside one another, the parser is free to build the tree either way.

- * Left recursion
- * Common prefix
- * Ambiguity.

Scenario : Operator Precedence Conflict

* The flat grammar $E \rightarrow E + E \mid E * E \mid (id)$ says nothing about the operators should be bind together, so it is ambiguous with respect to precedence. for the input $id + id * id$, the syntax analyzer can legally derive two different trees - one of the groups $(id + id)$

* Mathematically only for the second group $id + (id * id)$ is correct, because multiplication must be evaluated before addition. Since the grammar treats $+$ and $*$ as interchangeable in structure, the parser.

$$* \quad E \rightarrow E + T$$

$$E \rightarrow T * F$$

$$T \rightarrow T + F$$

$$T \rightarrow T * F$$

$$F \rightarrow id$$

$$\rightarrow S \rightarrow HIU$$

$$S \rightarrow U$$

$$H \rightarrow EIH$$

$$H \rightarrow H$$

$$U \rightarrow HES$$

$$U \rightarrow Hes.$$

The restructuring makes each string derivable in exactly one way. In practice most parser generation skip the grammar rewrite and instead apply the simpler convention of building.

2. Ambiguity in Conditional Statement

With the grammar $S \rightarrow \text{if } E \text{ then } S$
 $\text{if } E \text{ then } S \text{ else } S_1$, the syntax analyzer
meets ambiguity because a nested if-then
construct followed by a single else has more
than one valid parse tree - the else can be attached
to either the outer if or the inner one. This
is the classic dangling problem.

During parsing-table construction the conflict
show up because both production for S begin
with the identical prefix if E then ' S '. So the
first set overlaps completely - on seeing an else
in the input.

Decision: the ambiguity is resolved by
separating statement into those that are fully
matched the left unmatched grammar
in S, H, U

* Productions:

$$\rightarrow E \rightarrow E + E$$

$$E \rightarrow E * E$$

$$E \rightarrow id$$

While shift reduce reaches a state where neither a shift nor a reduce action is defined for the look ahead symbol, so the automation has no legal move. Both approaches correctly flag the error and the information generally

3. Scenarios Invalid Expression Handling:

* The lexical analyzer's job ends once it hand over a correctly classified token stream - for $a + * b$ each of a , $+$, $*$ and b is individually a legal token. The syntax analyzer's job.

* Different parsing strategies detect this the same ways but at different points a Predictive parser find no entry in the parsing table for the current non terminal / token pair and reports.

* Detection error handling should not stop at detection. The parser should incorporate error recovery strategies such as panic mode recovery, where the parser discard tokens until reaches a sync.

$$* E \rightarrow E + T$$

$$E \rightarrow E - T$$

$$E \rightarrow T$$

$$* T \rightarrow id$$

* Force the leftmost operator pair to be reduced first, guaranteeing left to right grouping. with this change the parser always produces $(id + id) - id$ for the given expression the tree



Seemingly Invariant parse tree generation

A top down or bottom up parser
analyzes build a parse tree by repeatedly
matching grammar productions to incoming
tokens stream. for the expression $id + id - id$
the shape of resulting tree.

When the grammar does not fix how
repeated operators of the same precedence
nest inside one another, the parser is free
to build the tree either way.

The root cause is that grammar's production
shape directly dictates tree structures: a symmetric
non recursive function and the grammar is
structurally ambiguous.